

LOW LEVEL DESIGN

The LLD Runbook

Ten rounds, from the first sentence to the follow-up.

Part 1 closed on the sentence this half exists to answer: the interviewer changes their mind, and both of you find out what your design is made of. Every case here runs to that moment, and then keeps going.

You get the prompt as it was worded, the questions I asked, the design I actually wrote — mistakes left in — and what happened when the requirement moved. Attempt each one before you read on. The value isn't in my answer; it's in finding where your instincts and mine part company.

Contents

1	Data structures under pressure	Two rounds I sat in
P1	LRU Cache Correct code, thirty minutes early, and the question that came after it.	3–6
P2	HashMap Storage and collisions, and which of the two the follow-up actually targets.	7–9
2	Objects, state and rules	Modelling and lifecycle
P3	Parking lot The classic. Where the nouns mislead you, and where the rules belong.	10–12
P4	Vending machine A state machine wearing a shopping cart's clothes.	13–15
P5	Elevator system Scheduling as a policy, not an algorithm you hard-code on the whiteboard.	16–18
P6	Snake and ladder Simple rules, until the board itself has to become data.	19–21
3	Services and policies	Extension under pressure
P7	Rate limiter One interface, four algorithms, and the state they refuse to share.	22–24
P8	Notification service Fan-out and retries, and the decision most candidates push one layer too low.	25–27
P9	Splitwise Money, invariants, and why a balance must never be stored twice.	28–30
P10	Logging framework The problem that looks like Decorator, and often isn't.	31–33
4	Revision	Reference
4.1	Pattern-to-problem cheat sheet	34
4.2	Where to go next	35

How to work a case

1 Attempt

Stop at the brief and design it yourself before turning the page.



2 Compare

Check ownership, invariants and one full flow — not class names.



3 Adapt

Answer the follow-up out loud before reading how I answered it.

P1 LRU Cache

The round I misread

The invite said low level design. The interviewer joined, shared a live code editor, and read out a problem that never used the word cache once. I had about a minute to decide what kind of round this was.

WHAT I WAS ASKED

"We show a user's location and address on the checkout page, and it needs to be near real time. The user base is large, so give priority to the people who come back often. We can afford to be a little slower for everyone else."

WHAT I SAID BACK

"This sounds like a cache. *Come back often* could mean lifetime visit count or recent activity — I read it as recency, so I'd start with least-recently-used eviction and a fixed capacity. Tell me if you meant frequency and I'll change the rule."

I drew nothing. I repeated the problem back, named the shape I thought it had, and put one assumption on the table — the opening move from section 6.1, more or less by reflex. That last clause is the part worth stealing: naming the interpretation you *rejected* costs four seconds and tells the interviewer you saw the ambiguity rather than missed it.

Q1 What am I actually building?

A couple of things needed pinning down before I wrote anything. I asked three, and only one of them changed the design.

Does a read count as a use? Yes. `get` promotes the key it finds.

Bounded by entries or by memory? Entries. Capacity is fixed when the cache is built, and never grows.

Does anything expire on its own? No. An entry leaves only when a new one needs its place.

The first row is the one that pays. If a read promotes the key, `get` is a mutating operation — and that single fact decides your data structure before you have drawn a box. The other two bound the problem without changing it.

We settled the rest out loud: single-threaded, exact recency rather than an approximation, generic keys and values. Expiry, persistence, cache warming and anything distributed were agreed out of scope. In a live-coding round, say the scope rather than typing it — the editor is for code, and a comment block of assumptions reads as stalling.

YOUR TURN

You now have exactly what I had: the prompt, three answers and the agreed scope. Give it fifteen minutes before you turn the page. Write down the classes you would keep, the public API, and one sentence on what you would allow to vary later. Not the implementation — the shape.

P1 LRU Cache

Two structures, one promise

Everything below follows from one promise: find any key in constant time, and move it to the front in constant time. Miss either half and this is just a slower map.

Q2 What are the things?

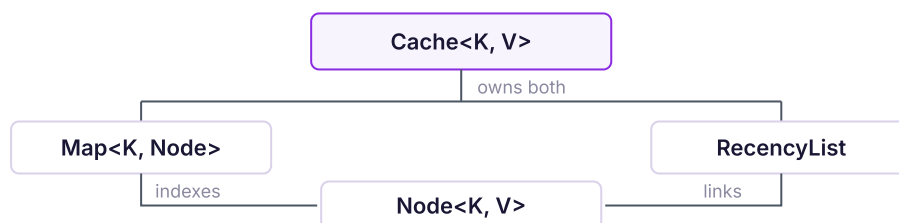
Three nouns from the prompt did not survive: `User`, `Address` and `CheckoutPage`. They are the reason the cache exists, not parts of it, and pulling them in would tie a general mechanism to one screen. Say your rejects out loud in the room — discarding a plausible noun scores better than listing it.

Q3 Who owns what?

What remained, and what each one is answerable for:

<code>Cache</code>	Owns both structures and leaves them consistent after every call. Nothing outside it touches either one.
<code>Map<K, Node></code>	Finds the node for a key. Mapping to values instead would force a scan to reorder.
<code>RecencyList</code>	Holds the order. Relinks a node it is handed, and never searches for one.
<code>Node</code>	Key, value, both links. It carries its own key so eviction can clean the index.

Two invariants hold the whole thing up: **every key has exactly one node, present in both structures**, and **size never exceeds capacity**. Name them while you design. Every bug in this problem is one of the two breaking.



The design as I wrote it. `Cache` decides which node leaves.

I wrote the signatures before any of the bodies. That is worth doing even when you are typing rather than drawing, because it forces the ownership above to be a decision rather than an accident.

THE PUBLIC SURFACE

```

public class LruCache<K, V> {
    private final Map<K, Node<K, V>> index;
    private final RecencyList<K, V> order;
    private final int capacity;

    public V get(K key) { /* promotes on a hit */ }
    public void put(K key, V value) { /* evicts when full */ }
}
  
```

Q4 What will change?

He asked what I thought would change. I said nothing — I had been asked for an LRU cache, so `put` takes the tail and there is nothing to vary. I used the time I had left to make the code clean.

P1 LRU Cache

What six calls proved

The code was done well before time. He wanted to see it run, so we walked inputs through it together in the editor rather than reading the implementation back.

Q5 Does it actually work?

Capacity three, starting empty:

CALL	RETURNS	ORDER, MOST RECENT FIRST	WHAT HAPPENED
<code>put(A, 1)</code>	—	A	Added to both structures.
<code>put(B, 2)</code>	—	B, A	New entry becomes head.
<code>put(C, 3)</code>	—	C, B, A	Full. Nothing evicted yet.
<code>get(A)</code>	1	A, C, B	A read reorders.
<code>put(D, 4)</code>	—	D, A, C	Tail B leaves both.
<code>put(A, 9)</code>	—	A, D, C	Update in place.

Rows four and six are the ones to rehearse. The fourth proves a read is a write, which is the clarification from the brief finally paying out. The sixth is the case candidates drop: overwriting a key that already exists must not create a node and must not evict anything. If your `put` inserts blindly, that is where it breaks, and it is usually the first thing an interviewer tests.

THE BRANCH ROW SIX EXERCISES

```
public void put(K key, V value) {
    Node<K, V> hit = index.get(key);
    if (hit != null) {
        hit.value = value;
        order.moveToFront(hit);
        return; // no new node, nothing evicted
    }
    if (index.size() == capacity) evictOne();
    index.put(key, order.addFront(key, value));
}
```

Derive the cost here, at the call site, rather than announcing Big-O at the end. Every line above is one hashed lookup or a fixed number of pointer writes, so `get` and `put` are both **O(1) average**, in **O(capacity)** space for nodes held by two structures at once. The worst case belongs to the hash function, not to this design — worth saying, because it shows you know which part you control.

The two bugs that show up here are both invariant breaks: a `get` that returns the value without reordering, which passes every test that ignores eviction order; and eviction that cleans one structure and forgets the other, which is exactly why `Node` carries its own key. An access-ordered `LinkedHashMap` avoids both in about five lines — the right answer in production, the wrong one here, because it hides the mechanism you were asked to demonstrate.

P1 LRU Cache

The question after the code worked

We finished the trace and the code was correct. That was the moment I relaxed, and the moment the round actually began, with about thirty minutes still on the clock.

THE FOLLOW-UP

“What happens to your code if tomorrow product wants it faster for the people who use it *least*, rather than the most recent ones?”

I panicked, then took two silent minutes — easily the best decision of that round. Section 6.2's four moves:

1. **Restated:** same storage, same API, a different entry chosen for eviction.
2. **Located:** one line inside `evictOne`, the one that takes the tail.
3. **Separated:** index, list and both $O(1)$ guarantees untouched; only the victim choice moves.
4. **Admitted:** the rule was hard-coded, so I named where it should have been swappable.

Making that one choice swappable is exactly what **Strategy** does — section 5.6's signal is one decision that keeps changing inside a settled object. Not State: the cache is handed its policy.

BEFORE — THE RULE IS A LINE OF CODE

```
private void evictOne() {
    Node<K, V> victim = order.last();           // "least recently used"
    order.remove(victim);
    index.remove(victim.key);
}
```

AFTER — THE RULE IS A COLLABORATOR

```
private void evictOne() {
    Node<K, V> victim = policy.select(order);   // tail for LRU, head for MRU
    order.remove(victim);
    index.remove(victim.key);
}
```

He asked what a third rule would cost. One new class, and four or five lines of wiring.



Dashed boxes arrived with the follow-up. A third rule is one more box on the same bar.

Say the limit out loud too: LFU fits this interface but not this structure, because it needs frequency counts the recency list never keeps. github.com/varunkashyap-vkd/lll-runbook/lru-cache

What he was probing: not whether I could write an LRU cache, but whether it would survive a product decision.

Practise next: make it thread-safe without breaking either invariant, then design LFU and name what has to change.

If you take one thing from this problem: a correct first version is evidence, not the finish line.